



Architecture et Administration des BDD

Présenté par:

Amal HALFAOUI (Epse GHERNAOUT)

Amal.halfaoui@univ-tlemcen.dz

amal.halfaoui@gmail.com

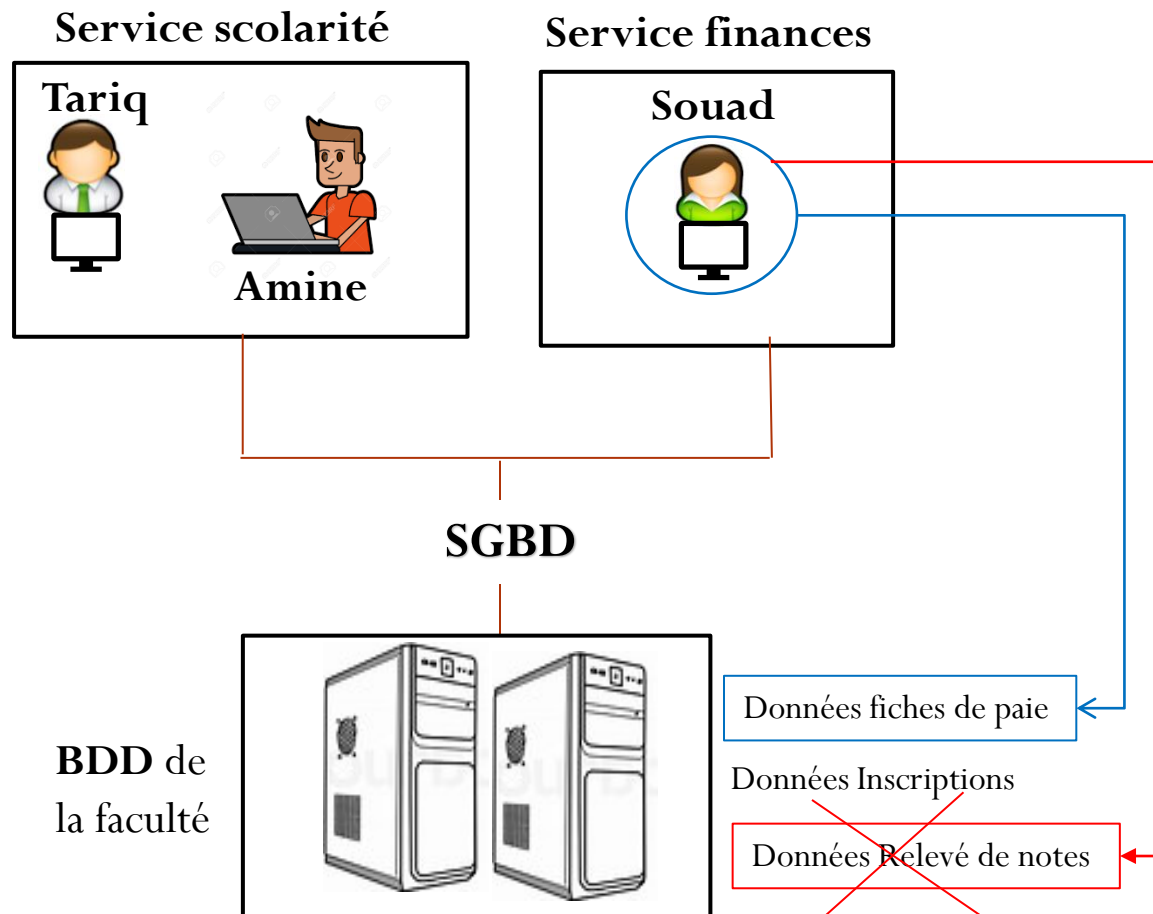
Université Tlemcen

-Ing 3-

-Les vues -

Rappel: Niveau externe

Niveau (schéma) externe



Rappel: Niveau externe

Niveau (schéma) externe

- Décrit comment un utilisateur ou programme perçoit les données
- Notion de **Vues** : Chaque utilisateur accède via la vue à une partie des données selon les droits d'accès associés
- Analyser les demandes de l'utilisateur et présentation des résultats
- Assurer la sécurité des données



Tariq

Schéma externe 1:
Vue scolarité

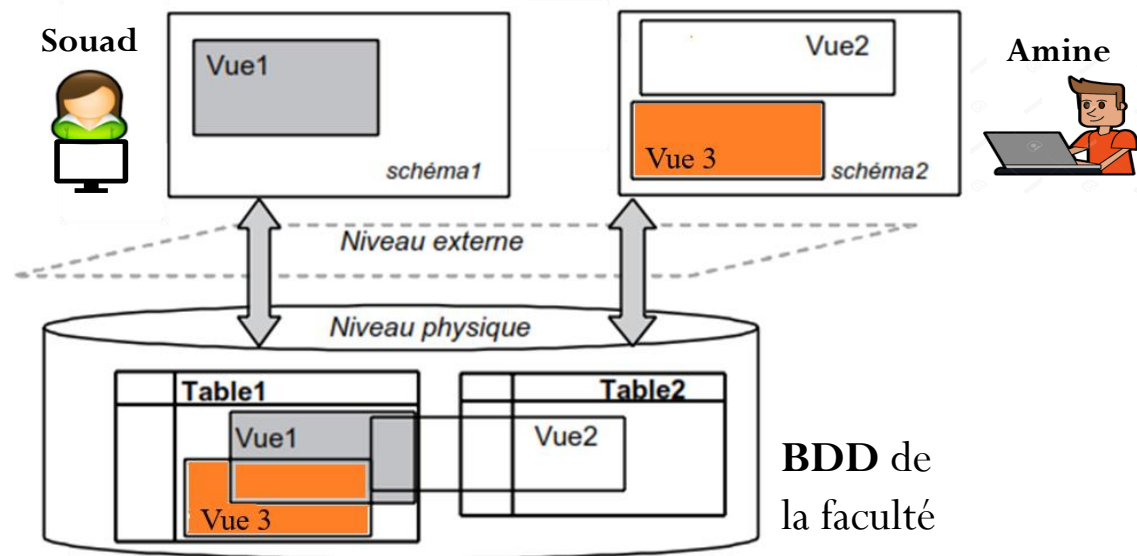


Souad

Schéma externe 2:
Vue finances

Les vues (Définition)

Est une vision logique des données contenues dans une ou plusieurs tables. Elle est considérée comme une table virtuelle qui n'a pas d'existence propre, aucune donnée ne lui est associée et sa description est stockée sous la forme d'une requête.



Les vues (Définition)

- Une vue est créée à l'aide d'une instruction SELECT appelée « **requête de définition** ». Cette requête interroge une (ou plusieurs) table(s) ou vue(s). Une vue se recharge chaque fois qu'elle est interrogée.
- La définition de la vues est stockée dans le dictionnaire de données , pas le résultat de la requête.

Utilité

- **Confidentialité**

Les utilisateurs n'ont le **droit d'accéder qu'aux vues** (Exp: vue sur la table Employé destinée aux employés ne sélectionne pas la colonne Salaire contrairement à la vue destinée aux chefs de service.)

- **Cacher la complexité des données aux utilisateurs**

Vue **simplifiée** des données

Simplifier les requêtes des utilisateurs non spécialistes

- **Présenter différentes perspectives sur les données aux utilisateurs**

Renommage des colonnes

- **Requêtes complexes**

Qui ne peuvent être faites sans vues

Qui sont fréquemment réalisées

Syntaxe

```
CREATE [OR REPLACE] VIEW nom_vue [ ( nv_nom_col)*]  
AS subquery  
[WITH CHECK OPTION [CONSTRAINT nom_contrainte] ]  
[WITH READ ONLY];
```

Exemple:

```
CREATE OR REPLACE VIEW Petit_pays  
AS SELECT * FROM Pays WHERE  
    surface < 100
```


Exemple 2

```
CREATE VIEW    PilotesAF
AS SELECT *    FROM Pilote
WHERE compa = 'AF';
```

Pilote

brevet	nom	nbHVol	adresse	compa
PL-1	Soutou	890	Castanet	CAST
PL-2	Laroche	500	Montauban	CAST
PL-3	Lamothe	1200	Ramonville	AF
PL-4	Albaric	500	Vieille-Toulouse	AF
PL-5	Bidal	120	Paris	ASO
PL-6	Labat	120	Pau	ASO
PL-7	Tauzin	100	Bas-Mauco	ASO

```
CREATE VIEW    Etat_civil
AS SELECT nom, nbvol, adresse, comp
From Pilote
```

Exemple 3

Etudiant:

ID_ETUDIANT	PRENOM	NOM	GENRE	DATE_NAISSANCE	EMAIL	NUM_TELEP...	DATE_INSCRIPTION	ID_SPECIALITE
2	Sara	Bouhadi	F	22/11/99	sara.bouhadi@etu.univ.edu	0605060708	02/09/22	2
3	Mohammed	Bakkali	M	10/07/98	mohammed.bakkali@etu.univ.edu	0609090909	03/09/22	3
1	Youssef	El Amrani	M	12/05/98	youssef.elamrani@etu.univ.edu	0601020304	01/09/22	1
4	Imane	Chraibi	F	18/03/00	imane.chraibi@etu.univ.edu	0610101010	04/09/22	1

Spécialité:

ID_SPECIALITE	NOM_SPECIALITE	ID_DEPARTEMENT
2	Intelligence artificielle	1
3	Réseaux	1
1	Génie logiciel	1

```
CREATE OR REPLACE VIEW ETUDIANTS_GL ("NOM", "PRENOM")  
AS select nom, prenom from etudiants e, specialites s  
WHERE e.id_specialite = s.id_specialite  
and nom_specialite = 'Génie logiciel';
```

Dictionnaire de donnée

- Informations sur la vue dans le dictionnaire de donnée

```
SELECT view_name, text  
FROM user_views
```

VIEW_NAME	TEXT
ETUDIANTS_GL	select nom, prenom from etudiants e, specialites s WHERE e.id_specialite = s.id

Interrogation d'une vue

- Une vue s'interroge **comme une table** par tout utilisateur, sous réserve qu'il ait obtenu le privilège en lecture directement (GRANT SELECT ON nomVue TO...)
- Exemples:

1- Les étudiants GL.

```
SELECT * FROM ETUDIANTS_GL
```

2-Somme des heures de vol des pilotes d'Air France.

```
SELECT SUM(nbHVol)  
FROM PilotesAF;
```

NOM	ENOM
-----	-----
Chraibi	Imane
El Amrani	Youssef

SUM(nbHVol)

1700.00

Vue d'une vue

- La requête de définition de la vue **porte sur une autre vue**.
- Exemple:

```
CREATE OR REPLACE VIEW    EtatCivilPilotesAF  
AS SELECT nomPil,heuresPil,adressePil  
      FROM PilotesAF
```

Classification d'une vue

- Une vue **monotable** est définie par une requête SELECT ne comportant qu'une seule table dans sa clause FROM.
- Une vue **multitable** comporte une jointure
- Une vue **simple** est une vue monotable qui ne comporte ni fonction d'agrégation , ni group by
- Une vue **complexe** est une vue multitable qui comporte des fonctions d'agrégation et/ou des groupements

Requête de définition	Vue simple	Vue complexe
Nombre de tables	1	1 ou plusieurs
Fonction	non	oui
Regroupement	non	oui
Mises à jour possibles ?	oui	pas toujours

Vues modifiables

- Lorsqu'il est possible d'exécuter des instructions **INSERT**, **UPDATE** ou **DELETE** sur une vue, cette dernière est dite « **modifiable** » (updatable view).
- Pour qu'une vue **simple soit modifiable**, sa requête de définition doit respecter les critères suivants :
 - pas de directive **DISTINCT**, de **fonction** (AVG, COUNT, MAX, MIN, SUM, ...) dans le SELECT ;
 - pas de **GROUP BY**, **ORDER BY** ou **HAVING**.

Vues modifiables

- Quand la vue est modifiable, les modifications faites à travers la vue impliquent **physiquement** la table de la requête de définition de la vue
- Exemple

```
INSERT INTO PilotesAF VALUES ('PL-10','CALABI', 200,'TOULOUSE','AF');
```

L'enregistrement est ajoutée a la table Pilote

Vues modifiables

- Exemples:

```
DELETE FROM piloteAF  
WHERE compa = 'ASO';
```

```
UPDATE Etat_civil  
SET nbHVol = nbHVol*2  
WHERE nom = 'Lamothe';
```

Insertion n'est pas conseillée quand le champs clé n'est pas renseigné sinon pas possible

```
INSERT INTO Etat_civil VALUES ('Raffarin',10,'Poitiers','ASO') ;
```

Attention : Ajout d'un pilote qui n'est pas de 'AF' !

```
INSERT INTO PilotesAF VALUES ('PL-9','Caboche', 600,'Rennes','ASO');
```

Vues complexes modifiables!!

- Une vue complexe est caractérisée par le fait qu'elle contient, dans sa définition, **plusieurs tables** (jointures) et une **fonction** appliquée à des regroupements ou à des expressions. La mise à jour de telles vues **n'est pas toujours possible**.
- Pour pouvoir modifier à travers une vue complexe, les restrictions sont les suivantes:
 - La requête de définition ne doit pas contenir de **sous-interrogation** (jointure procédurale).
 - Il n'est pas possible d'utiliser d'opérateur ensembliste (sauf UNION [ALL]).
 - Une des tables de la jointure peut être modifiée **si** sa clé est **préservée** dans le select de la jointure (**peut jouer le rôle de clé primaire de la vue**).

Vues complexes modifiables!!

- Pour connaître les opérations permises en modification des colonnes d'une vue (insertion, modification ou suppression), il faut interroger la vue **USER_UPDATABLE_COLUMNS** du dictionnaire de donnée

```
SELECT COLUMN_NAME, INSERTABLE,  
UPDATABLE, DELETABLE  
FROM USER_UPDATABLE_COLUMNS  
WHERE TABLE_NAME = UPPER('nom de la vue');
```

Vues complexes modifiables!! Exemple

Table pilote est préservée ou protégée par la clé **brevet**

```
CREATE VIEW Pilotes_multi_AF
AS SELECT p.brevet, p.nom, p.nbHVol,
          c.ville, c.nomComp
FROM Pilote p, Compagnie c
WHERE p.compa = c.comp AND p.compa = 'AF';
```

Compagnie

comp	nrue	rue	ville	nomComp
AF	124	Port Royal	Paris	Air France
SING	7	Camparols	Singapour	Singapore AL

Pilote

brevet	nom	nbHVol	compa
PL-1	Louise Ente	450	AF
PL-2	Jules Ente	900	AF
PL-3	Paul Soutou	1000	SING

Non modifiable

```
CREATE VIEW Moyenne_Heures_Pil
AS SELECT compa,
          AVG(nbHVol) moyenne
FROM Pilote GROUP BY compa;
```

compa	moyenne
AF	675
SING	1000

Vues complexes modifiables!! Exemple

```
SELECT * from vue_Multi_Comp_Pil;
```

COMP	NOMCOMP	BREVET	NOM	NBHVOL
AF	Air France	PL-1	Amélie Sulpice	2700
AF	Air France	PL-2	Thomas Sulpice	5400
SING	Singapore AL	PL-3	Paul Soutou	3000

Brevet peut
jouer le rôle
de clé de la
vue

```
SELECT COLUMN_NAME, INSERTABLE,  
UPDATABLE, DELETABLE FROM USER_UPDATABLE_COLUMNS  
WHERE TABLE_NAME = UPPER('vue_Multi_Comp_Pil')
```

COLUMN_NAME	INSERTABLE	UPDATABLE	DELETABLE
COMP	NO	NO	NO
NOMCOMP	NO	NO	NO
BREVET	YES	YES	YES
NOM	YES	YES	YES
NBHVOL	YES	YES	YES

Les colonnes de
la table pilote
sont modifiables

Directive CHECK OPTION (contraintes)

- La mise à jour n'est pas possible si l'opération ne respecte pas la restriction de la requête de définition de la vue
- Exemples

```
CREATE OR REPLACE VIEW PilotesAF  
AS SELECT * FROM pilote where comp='AF'  
WITH CHECK OPTION;
```

```
INSERT INTO PilotesAF VALUES ('PL-9','Caboche',600,'Rennes','ASO');
```

ERROR 1369 (HY000): CHECK OPTION failed 'PilotesAF'

```
CREATE OR REPLACE VIEW Petit_pays  
AS SELECT * FROM Pays WHERE surface < 100  
WITH CHECK OPTION CONSTRAINT CK_Petit_pays;
```

```
INSERT INTO Petit_pays values ("Chine", "Pékin", 2000, 500)
```

→ Impossible!!! Car surface est égale à 500

```
INSERT INTO Petit_pays values ("Pays", « Ville", 2000, 90)
```

→ Possible

Directive READ ONLY (constraints)

- Aucune opération n'est possible à travers la vue
- Exemple

```
CREATE OR REPLACE VIEW Petit_pays  
AS SELECT * FROM Pays WHERE surface < 100  
WITH READ ONLY;
```

INSERT INTO Petit_pays values ("Pays", « Ville", 2000, 90) → Impossible!!! Même si la surface est 100

Confidentialité

La confidentialité est une des vocations premières des vues. Outre l'utilisation de variables d'environnement, il est possible de restreindre l'accès à des tables en fonction de moments précis.

```
CREATE VIEW
VueDesCompagniesJoursFeries
AS SELECT * FROM Compagnie
WHERE DATE_FORMAT(SYSDATE(), '%W')
      IN ('Sunday', 'Saturday')
```

Restriction en lecture de la table Compagnie, les samedi et dimanches. Mises à jour possibles à tout moment.

```
CREATE VIEW
VueDesPilotesJoursOuvrables
AS SELECT * FROM Pilote
WHERE CURTIME()+0 BETWEEN 83000
AND 173000
AND DATE_FORMAT(SYSDATE(), '%W')
NOT IN ('Sunday', 'Saturday')
WITH CHECK OPTION;
```

Restriction, en lecture et en écriture (à cause de WITH CHECK OPTION), de la table Pilote les jours ouvrables de 8h30 à 17h30.

Attribution des droits

```
Create view Information_me_concernant  
As select *  
From employe  
Where upper(nom) = upper (user)
```

Chaque utilisateur peut visualiser un seul tuple de la relation: celui qui le concerne



```
GRANT SELECT ON Information_me_concernant TO public
```

```
GRANT insert ON VueDesCompagniesJoursFeries TO Tariq;
```

Suppression et renommage

- Supprimer une vue :

```
DROP VIEW nomvue [, ...] [CASCADE |  
RESTRICT ] ;
```

- **CASCADE** : Supprime automatiquement les objets qui dépendent de la vue (comme par exemple d'autres vues).
- **RESTRICT** : Refuse de supprimer la vue si un objet en dépend. Ceci est la valeur par défaut.

- Renommer une vue :

```
RENAME <ancien nom de vue> TO <nouveau nom  
de vue>
```

Trigger sur des vues

- Sous Oracle, triggers **INSTEAD OF**. (détaillé dans le chapitre des triggers)
- Les triggers INSTEAD OF permettent d'effectuer des modifications lorsque la vue est non modifiable.

Référence

- Livre SQL pour Oracle, de C.Soutou.